

A Finite Modulo 216 Certificate for a Counterexample to Hsia's 1981 Conjecture

20 August 2026

Abstract

In 1981, Hsia conjectured that two positive definite integral ternary quadratic forms in the same genus must be integrally equivalent whenever they primitively represent the same positive integers. We give a computer-assisted disproof using the pair

$$\begin{aligned}q_1(x, y, z) &= 5x^2 + 5y^2 + 8z^2 - 4xz - 3xy, \\q_2(x, y, z) &= 5x^2 + 7y^2 + 7z^2 + 6yz + xz + 5xy.\end{aligned}$$

These are the forms in the set S_8 of Ju. Their equality of ordinary representation sets was proved by Ju, while a computation of Jagy, as reported by Ju, verified equality of their primitive representation sets through 10^6 . Here a finite exact certificate upgrades the latter to all positive integers. The certificate consists of 68 integral similitudes in each direction, with denominators in $\{4, 12, 36\}$, and an exhaustive cover of the 8,491,392 residue vectors modulo 216 that are primitive at both 2 and 3. A stand-alone C program, reproduced in full, verifies every matrix identity, determinant, and residue-class condition using only integer arithmetic.

2020 Mathematics Subject Classification. 11E12, 11E20, 11Y40.

Keywords. ternary quadratic form, primitive representation, genus, finite certificate, computer-assisted proof.

1 Introduction and statement of the result

For a positive definite integral ternary quadratic form q , write

$$\text{Prim}(q) = \{n \in \mathbf{Z}_{>0} : q(v) = n \text{ for some } v \in \mathbf{Z}^3 \text{ with } \gcd(v_1, v_2, v_3) = 1\}.$$

Two forms are *integrally equivalent* if one is carried to the other by an element of $\text{GL}_3(\mathbf{Z})$, and they lie in the same genus if they are equivalent over $\mathbf{R}$ and over $\mathbf{Z}_p$ for every prime p .

Hsia's Conjecture A, formulated in [1, p. 237], asserts that positive definite ternary forms in the same genus are integrally equivalent if they primitively represent the same integers. The pair studied here appears as S_8 in Ju's table [2, Table 1.1]. Ju proved that the two forms represent the same integers without a primitivity condition and recorded Jagy's verification of primitive agreement up to 1,000,000 [2, Remark 3.1(2)].

Theorem 1.1. *Let*

$$\begin{aligned} q_1(x, y, z) &= 5x^2 + 5y^2 + 8z^2 - 4xz - 3xy, \\ q_2(x, y, z) &= 5x^2 + 7y^2 + 7z^2 + 6yz + xz + 5xy. \end{aligned}$$

Then:

- (i) q_1 and q_2 are positive definite and belong to the same genus;
- (ii) q_1 and q_2 are not integrally equivalent;
- (iii) $\text{Prim}(q_1) = \text{Prim}(q_2)$.

Consequently, Hsia's 1981 conjecture is false.

The proof of Theorem 1.1 has a short conceptual part and a finite certificate. The conceptual part is isolated in the primitive transfer lemma of Section 3. The complete certificate is the source code in Section B; it includes all 136 matrices and checks all relevant residue classes. No floating-point calculation, random test, external library, or unproved bound is used.

Remark 1.2 (Status and scope). This note records an explicit, independently rerunnable certificate. The forms themselves and their previously known properties are due to the literature cited above. No claim of priority is made here beyond those references; the mathematical assertion of the note is the finite global certificate and the deduction from it.

2 The two forms

Set

$$F = \begin{pmatrix} 10 & -3 & -4 \\ -3 & 10 & 0 \\ -4 & 0 & 16 \end{pmatrix}, \quad G = \begin{pmatrix} 10 & 5 & 1 \\ 5 & 14 & 6 \\ 1 & 6 & 14 \end{pmatrix}.$$

Thus, for column vectors $v = (x, y, z)^\top$,

$$2q_1(v) = v^\top F v, \quad 2q_2(v) = v^\top G v.$$

2.1 Positive definiteness and the genus

Proposition 2.1. *The forms q_1 and q_2 are positive definite and lie in the same genus.*

Proof. The leading principal minors of F are

$$10, \quad 91, \quad 1296,$$

and those of G are

$$10, \quad 115, \quad 1296.$$

They are all positive, so Sylvester's criterion gives positive definiteness.

Now put

$$T_3 = \begin{pmatrix} -3 & -2 & -2 \\ 0 & -3 & -3 \\ 0 & 1 & -2 \end{pmatrix}, \quad T_4 = \begin{pmatrix} -4 & 0 & 0 \\ 0 & 4 & 4 \\ -2 & -2 & 2 \end{pmatrix}.$$

Direct multiplication gives

$$T_3^\top F T_3 = 3^2 G, \quad \det(T_3) = -3^3,$$

and

$$T_4^\top F T_4 = 4^2 G, \quad \det(T_4) = -4^3.$$

Hence $T_3/3 \in \mathrm{GL}_3(\mathbf{Z}_2)$ carries F to G . For every odd prime p , the matrix $T_4/4$ belongs to $\mathrm{GL}_3(\mathbf{Z}_p)$ and carries F to G . The same rational matrix gives an equivalence over $\mathbf{R}$. Thus the two forms are locally equivalent at every place of $\mathbf{Q}$. $\square$

2.2 Integral inequivalence

For a positive definite form q , let

$$r_q(n) = \#\{v \in \mathbf{Z}^3 : q(v) = n\}.$$

Integral equivalence preserves every $r_q(n)$.

Proposition 2.2. *The forms q_1 and q_2 are not integrally equivalent.*

Proof. The four vectors

$$(\pm 1, 0, 0), \quad (0, \pm 1, 0)$$

represent 5 under q_1 , so $r_{q_1}(5) \geq 4$.

Suppose $q_2(x, y, z) = 5$. Treating this as a quadratic equation in x gives the discriminant

$$\Delta = 100 - (115y^2 + 110yz + 139z^2).$$

The quadratic expression in parentheses can be written as

$$115y^2 + 110yz + 139z^2 = 55(y + z)^2 + 60y^2 + 84z^2.$$

For $(y, z) \neq (0, 0)$ it is at least 115, so $\Delta < 0$. Therefore $y = z = 0$, and then $x = \pm 1$. Thus $r_{q_2}(5) = 2$, proving the claim. $\square$

3 A primitive transfer lemma

For a symmetric integral matrix A , write

$$Q_A(v) = \frac{1}{2} v^\top A v.$$

The following elementary observation is the bridge between the matrix certificate and primitive representations.

Lemma 3.1 (Primitive transfer). *Let $A, B \in \mathrm{Sym}_3(\mathbf{Z})$ be nonsingular, let $d \geq 1$, and let $T \in M_3(\mathbf{Z})$ satisfy*

$$T^\top A T = d^2 B, \quad |\det T| = d^3.$$

Let $v \in \mathbf{Z}^3$ be primitive. Assume that $Tv \in d\mathbf{Z}^3$ and put $w = Tv/d$. If $w \notin p\mathbf{Z}^3$ for every prime $p \mid d$, then w is primitive and

$$Q_A(w) = Q_B(v).$$

Proof. The quadratic identity is immediate:

$$2Q_A(w) = \frac{1}{d^2} v^\top T^\top A T v = v^\top B v = 2Q_B(v).$$

Suppose a prime p divides all coordinates of w . By hypothesis this cannot happen when $p \mid d$. If $p \nmid d$, then

$$T v = d w \equiv 0 \pmod{p}.$$

But $\det T = \pm d^3$ is nonzero modulo p , so T is invertible modulo p and hence $v \equiv 0 \pmod{p}$, contradicting the primitivity of v . Thus no prime divides all coordinates of w . $\square$

4 The finite certificate

Let

$$\mathcal{D} = \{4, 12, 36\}, \quad L = 216.$$

For each $d \in \mathcal{D}$, the certificate contains two finite families

$$\mathcal{T}_{G \rightarrow F}(d), \quad \mathcal{T}_{F \rightarrow G}(d)$$

of integral 3×3 matrices. Their cardinalities are as follows.

Direction	$d = 4$	$d = 12$	$d = 36$	Total
$G \rightarrow F$	4	16	48	68
$F \rightarrow G$	4	16	48	68

The matrices are listed exactly in the C arrays `GF4`, `GF12`, `GF36`, `FG4`, `FG12`, and `FG36` in Section B. The checker verifies

$$T^\top F T = d^2 G, \quad |\det T| = d^3 \quad \text{for } T \in \mathcal{T}_{G \rightarrow F}(d),$$

and

$$T^\top G T = d^2 F, \quad |\det T| = d^3 \quad \text{for } T \in \mathcal{T}_{F \rightarrow G}(d).$$

Define

$$\mathcal{R}_L = \left\{ \bar{v} \in (\mathbf{Z}/L\mathbf{Z})^3 : \bar{v} \not\equiv 0 \pmod{2}, \bar{v} \not\equiv 0 \pmod{3} \right\},$$

where $\bar{v} \equiv 0 \pmod{p}$ means that all three coordinates vanish modulo p . Its size is

$$\begin{aligned} |\mathcal{R}_L| &= 216^3 - 108^3 - 72^3 + 36^3 \\ &= 8,491,392. \end{aligned}$$

Proposition 4.1 (Finite cover). *For every $\bar{v} \in \mathcal{R}_L$, in each of the two directions there exist $d \in \mathcal{D}$ and a matrix T in the corresponding family such that, for any lift $v \in \mathbf{Z}^3$ of $\bar{v}$,*

$$T v \in d \mathbf{Z}^3, \tag{1}$$

$$\frac{T v}{d} \notin p \mathbf{Z}^3 \quad \text{for every prime } p \mid d. \tag{2}$$

Certificate verification. Condition (1) depends only on v modulo d . Once it holds, condition (2) for a prime $p \mid d$ depends only on v modulo dp . The only moduli that occur are

$$8, 24, 36, 72, 108,$$

and their least common multiple is 216. Therefore it is enough to check every element of $\mathcal{R}_L$.

The function `test_union` in Section B performs exactly this exhaustive check in both directions. It reports no uncovered residue class. The complete output is reproduced in Section A. $\square$

Remark 4.2. The modulus 216 is a logical completeness modulus, not a numerical cutoff for represented integers. Once Proposition 4.1 is verified, it applies to every primitive vector in $\mathbf{Z}^3$.

5 Proof of the main theorem

Proof of Theorem 1.1. Part (i) is Proposition 2.1, and part (ii) is Proposition 2.2. It remains to prove part (iii).

Let $n \in \text{Prim}(q_2)$. Choose a primitive vector $v \in \mathbf{Z}^3$ with $q_2(v) = n$. Since v is primitive, its residue class modulo 216 lies in $\mathcal{R}_L$. Apply Proposition 4.1 in the $G \rightarrow F$ direction. We obtain $d \in \{4, 12, 36\}$ and $T \in \mathcal{T}_{G \rightarrow F}(d)$ such that

$$w = \frac{Tv}{d} \in \mathbf{Z}^3$$

is primitive at every prime dividing d . By Lemma 3.1, w is globally primitive and

$$q_1(w) = q_2(v) = n.$$

Thus $\text{Prim}(q_2) \subseteq \text{Prim}(q_1)$.

The same argument with the $F \rightarrow G$ families gives the reverse inclusion. Hence

$$\text{Prim}(q_1) = \text{Prim}(q_2).$$

Together with the genus equality and integral inequivalence, this gives a counterexample to Hsia's conjecture. $\square$

6 Reproducibility and arithmetic safety

The certificate can be compiled and run with any conforming C11 compiler:

```
cc -O3 -std=c11 -Wall -Wextra -pedantic \
    hsia_primitive_certificate.c -o hsia_primitive_certificate
./hsia_primitive_certificate
```

A failed identity or uncovered residue class increments the variable `fail`; the program exits with nonzero status unless the final line is `RESULT PASS`.

All arithmetic is far inside the range of a signed 64-bit integer. The largest absolute matrix entry in the certificate is 48, the largest absolute Gram-matrix entry is 16, and residue coordinates lie between 0 and 215. Consequently:

- a coordinate of Tv has absolute value at most $3 \cdot 48 \cdot 215 = 30,960$;

- an accumulator for an entry of $T^T A T$ has absolute value at most $9 \cdot 48 \cdot 16 \cdot 48 = 331,776$;
- determinant computations are also below 10^6 in absolute value.

Thus the checker does not rely on implementation-dependent overflow.

The SHA-256 digests of the source file and the verification transcript distributed with this note are, respectively,

```
6f668e2fd9722ce3b6e91761b626667c0f5f3976582e8c6352a99db2d7070acb
54d123643d6476843465f996e54c3293927be15c7661f6d025e80c2fcc443440
```

A Verification transcript

The following is the complete output of the checker.

```
VERIFY T3   matrices=1 fail=0
VERIFY T4   matrices=1 fail=0
VERIFY GF4  matrices=4 fail=0
VERIFY GF12 matrices=16 fail=0
VERIFY GF36 matrices=48 fail=0
VERIFY FG4  matrices=4 fail=0
VERIFY FG12 matrices=16 fail=0
VERIFY FG36 matrices=48 fail=0
COVER direction=G_to_F residues=8491392 fail=0
COVER direction=F_to_G residues=8491392 fail=0
RESULT PASS
```

B Complete stand-alone C certificate

The source below is part of the proof: it contains all explicit matrices and the exhaustive finite-cover verification.

Listing 1: Exact certificate for the primitive representation equivalence.

```
1  /*
2  Exact certificate for a counterexample to Hsia's 1981 Conjecture A.
3
4  Quadratic forms:
5       $q1(x,y,z) = 5x^2 + 5y^2 + 8z^2 - 4xz - 3xy,$ 
6       $q2(x,y,z) = 5x^2 + 7y^2 + 7z^2 + 6yz + xz + 5xy.$ 
7
8  The integral matrices F and G below satisfy
9       $2 \ q1(v) = v^T F v, \quad 2 \ q2(v) = v^T G v.$ 
10
11 For each listed T in the G-to-F families,
12  $T^T F T = d^2 G$  and  $|\det T| = d^3.$ 
13 For each listed T in the F-to-G families,
14  $T^T G T = d^2 F$  and  $|\det T| = d^3.$ 
15
16 The program checks every residue vector modulo 216 that is primitive at 2
17 and 3. In both directions it finds d in {4,12,36} and a listed T such that
18  $w = T v / d$  is integral and primitive at every prime dividing d. Primitivity at
19 all other primes follows from  $\det(T) = \pm d^3$ . Thus q1 and q2 have identical
20 primitive representation sets.
21
22 Compile and run, for example:
```

```

23 cc -O3 -std=c11 -Wall -Wextra -pedantic \
24     hsia_1981_counterexample_certificate.c -o hsia_cert
25     ./hsia_cert
26
27 No external library is used.
28 */
29 #include <stdio.h>
30 #include <stdint.h>
31
32 static const int G[3][3]={10,5,1},{5,14,6},{1,6,14}};
33 static const int F[3][3]={10,-3,-4},{-3,10,0},{-4,0,16}};
34
35 /*  $T_3 \sim F T_3 = 9G$  and  $T_4 \sim F T_4 = 16G$ .
36     $T_3/3$  gives the 2-adic equivalence;  $T_4/4$  gives every odd-adic equivalence. */
37 static const int T3[3][3]={-3,-2,-2},{0,-3,-3},{0,1,-2}};
38 static const int T4[3][3]={-4,0,0},{0,4,4},{-2,-2,2}};
39
40 static const int GF4[4][3][3]={
41     {{-4,0,0},{0,4,4},{-2,-2,2}},
42     {{-4,0,0},{0,4,4},{0,2,-2}},
43     {{4,0,0},{0,-4,-4},{0,-2,2}},
44     {{4,0,0},{0,-4,-4},{2,2,-2}},
45 };
46
47 static const int GF12[16][3][3]={
48     {{-12,-8,-8},{0,-12,-12},{-6,-8,4}},
49     {{-12,0,0},{0,12,12},{-6,-6,6}},
50     {{-12,-8,-8},{0,-12,-12},{0,4,-8}},
51     {{-12,0,0},{0,12,12},{0,6,-6}},
52     {{-4,-12,4},{0,-12,-12},{-10,-6,-2}},
53     {{-4,-4,12},{0,12,12},{-10,-4,0}},
54     {{-4,-12,4},{0,-12,-12},{8,0,4}},
55     {{-4,-4,12},{0,12,12},{8,2,6}},
56     {{4,4,-12},{0,-12,-12},{-8,-2,-6}},
57     {{4,12,-4},{0,12,12},{-8,0,-4}},
58     {{4,4,-12},{0,-12,-12},{10,4,0}},
59     {{4,12,-4},{0,12,12},{10,6,2}},
60     {{12,0,0},{0,-12,-12},{0,-6,6}},
61     {{12,8,8},{0,12,12},{0,-4,8}},
62     {{12,0,0},{0,-12,-12},{6,6,-6}},
63     {{12,8,8},{0,12,12},{6,8,-4}},
64 };
65
66 static const int GF36[48][3][3]={
67     {{-40,-20,-4},{-12,-24,24},{-10,-32,-28}},
68     {{-40,-20,-4},{-12,-24,24},{-10,22,26}},
69     {{-36,-24,-24},{0,-36,-36},{-18,-24,12}},
70     {{-36,0,0},{0,36,36},{-18,-18,18}},
71     {{-36,-24,-24},{0,-36,-36},{0,12,-24}},
72     {{-36,0,0},{0,36,36},{0,18,-18}},
73     {{-32,-4,-20},{12,24,-24},{-8,-28,-32}},
74     {{-32,-4,-20},{12,24,-24},{-8,26,22}},
75     {{-24,-28,20},{-36,-12,-12},{-6,-34,-22}},
76     {{-24,-28,20},{-36,-12,-12},{-6,20,32}},
77     {{-12,-40,-40},{-36,-12,-12},{-12,-28,8}},
78     {{-12,32,32},{-36,-12,-12},{-12,-10,26}},
79     {{-12,-40,-40},{-36,-12,-12},{6,8,-28}},
80     {{-12,32,32},{-36,-12,-12},{6,26,-10}},
81     {{-12,-36,12},{0,-36,-36},{-30,-18,-6}},
82     {{-12,-12,36},{0,36,36},{-30,-12,0}},
83     {{-12,-36,12},{0,-36,-36},{24,0,12}},
84     {{-12,-12,36},{0,36,36},{24,6,18}},
85     {{-4,-44,-28},{-12,-24,24},{-28,-20,-16}},
86     {{-4,28,44},{-12,-24,24},{-28,-2,2}},
87     {{-4,-44,-28},{-12,-24,24},{26,-2,2}},
88     {{-4,28,44},{-12,-24,24},{26,16,20}},
89     {{0,20,-28},{-36,-12,-12},{0,-22,-34}},

```

```

90  {{0,20,-28},{-36,-12,-12},{0,32,20}},
91  {{0,-20,28},{36,12,12},{0,-32,-20}},
92  {{0,-20,28},{36,12,12},{0,22,34}},
93  {{4,-28,-44},{12,24,-24},{-26,-16,-20}},
94  {{4,44,28},{12,24,-24},{-26,2,-2}},
95  {{4,-28,-44},{12,24,-24},{28,2,-2}},
96  {{4,44,28},{12,24,-24},{28,20,16}},
97  {{12,12,-36},{0,-36,-36},{-24,-6,-18}},
98  {{12,36,-12},{0,36,36},{-24,0,-12}},
99  {{12,12,-36},{0,-36,-36},{30,12,0}},
100 {{12,36,-12},{0,36,36},{30,18,6}},
101 {{12,-32,-32},{36,12,12},{-6,-26,10}},
102 {{12,40,40},{36,12,12},{-6,-8,28}},
103 {{12,-32,-32},{36,12,12},{12,10,-26}},
104 {{12,40,40},{36,12,12},{12,28,-8}},
105 {{24,28,-20},{36,12,12},{6,-20,-32}},
106 {{24,28,-20},{36,12,12},{6,34,22}},
107 {{32,4,20},{-12,-24,24},{8,-26,-22}},
108 {{32,4,20},{-12,-24,24},{8,28,32}},
109 {{36,0,0},{0,-36,-36},{0,-18,18}},
110 {{36,24,24},{0,36,36},{0,-12,24}},
111 {{36,0,0},{0,-36,-36},{18,18,-18}},
112 {{36,24,24},{0,36,36},{18,24,-12}},
113 {{40,20,4},{12,24,-24},{10,-22,-26}},
114 {{40,20,4},{12,24,-24},{10,32,28}},
115 };
116
117 static const int FG4[4][3][3]={
118 {{-4,0,0},{2,2,-4},{-2,2,4}},
119 {{-4,0,0},{0,2,4},{0,2,-4}},
120 {{4,0,0},{0,-2,-4},{0,-2,4}},
121 {{4,0,0},{-2,-2,4},{2,-2,-4}},
122 };
123
124 static const int FG12[16][3][3]={
125 {{-12,8,0},{6,-8,-12},{-6,-4,12}},
126 {{-12,0,0},{6,6,-12},{-6,6,12}},
127 {{-12,8,0},{0,-8,12},{0,-4,-12}},
128 {{-12,0,0},{0,6,12},{0,6,-12}},
129 {{4,4,-16},{-10,-4,4},{10,-8,-4}},
130 {{4,-4,-16},{-10,10,4},{10,2,-4}},
131 {{-4,4,16},{-8,-4,-4},{8,-8,4}},
132 {{-4,-4,16},{-8,10,-4},{8,2,4}},
133 {{4,4,-16},{8,-10,4},{-8,-2,-4}},
134 {{4,-4,-16},{8,4,4},{-8,8,-4}},
135 {{-4,4,16},{10,-10,-4},{-10,-2,4}},
136 {{-4,-4,16},{10,4,-4},{-10,8,4}},
137 {{12,0,0},{0,-6,-12},{0,-6,12}},
138 {{12,-8,0},{0,8,-12},{0,4,12}},
139 {{12,0,0},{-6,-6,12},{6,-6,-12}},
140 {{12,-8,0},{-6,8,12},{6,4,-12}},
141 };
142
143 static const int FG36[48][3][3]={
144 {{-40,12,16},{16,-30,-28},{-4,30,-20}},
145 {{-32,12,-16},{2,-30,28},{-14,30,20}},
146 {{-36,24,0},{18,-24,-36},{-18,-12,36}},
147 {{-36,0,0},{18,18,-36},{-18,18,36}},
148 {{-36,24,0},{0,-24,36},{0,-12,-36}},
149 {{-36,0,0},{0,18,36},{0,18,-36}},
150 {{-40,12,16},{16,24,-28},{-4,-24,-20}},
151 {{-32,12,-16},{2,24,28},{-14,-24,20}},
152 {{-4,-36,16},{-20,18,-28},{32,-18,-20}},
153 {{4,-36,-16},{-34,18,28},{22,-18,20}},
154 {{12,-40,0},{-12,16,-36},{-24,-4,36}},
155 {{-12,-32,0},{30,2,-36},{6,-14,36}},
156 {{12,-40,0},{-30,16,36},{-6,-4,-36}},

```



```

157 {-12,-32,0},{12,2,36},{24,-14,-36}},
158 {{12,12,-48},{-30,-12,12},{30,-24,-12}},
159 {{12,-12,-48},{-30,30,12},{30,6,-12}},
160 {{-12,12,48},{-24,-12,-12},{24,-24,12}},
161 {{-12,-12,48},{-24,30,-12},{24,6,12}},
162 {{24,-4,-48},{-24,-20,12},{-12,32,-12}},
163 {{0,4,-48},{18,-34,12},{18,22,-12}},
164 {{0,-4,48},{-18,-20,-12},{-18,32,12}},
165 {{-24,4,48},{24,-34,-12},{12,22,12}},
166 {{-4,-36,16},{34,0,-28},{-22,0,-20}},
167 {{4,-36,-16},{20,0,28},{-32,0,20}},
168 {{-4,36,16},{-20,0,-28},{32,0,-20}},
169 {{4,36,-16},{-34,0,28},{22,0,20}},
170 {{24,-4,-48},{-24,34,12},{-12,-22,-12}},
171 {{0,4,-48},{18,20,12},{18,-32,-12}},
172 {{0,-4,48},{-18,34,-12},{-18,-22,12}},
173 {{-24,4,48},{24,20,-12},{12,-32,12}},
174 {{12,12,-48},{24,-30,12},{-24,-6,-12}},
175 {{12,-12,-48},{24,12,12},{-24,24,-12}},
176 {{-12,12,48},{30,-30,-12},{-30,-6,12}},
177 {{-12,-12,48},{30,12,-12},{-30,24,12}},
178 {{12,32,0},{-12,-2,-36},{-24,14,36}},
179 {{-12,40,0},{30,-16,-36},{6,4,36}},
180 {{12,32,0},{-30,-2,36},{-6,14,-36}},
181 {{-12,40,0},{12,-16,36},{24,4,-36}},
182 {{-4,36,16},{34,-18,-28},{-22,18,-20}},
183 {{4,36,-16},{20,-18,28},{-32,18,20}},
184 {{32,-12,16},{-2,-24,-28},{14,24,-20}},
185 {{40,-12,-16},{-16,-24,28},{4,24,20}},
186 {{36,0,0},{0,-18,-36},{0,-18,36}},
187 {{36,-24,0},{0,24,-36},{0,12,36}},
188 {{36,0,0},{-18,-18,36},{18,-18,-36}},
189 {{36,-24,0},{-18,24,36},{18,12,-36}},
190 {{32,-12,16},{-2,30,-28},{14,-30,-20}},
191 {{40,-12,-16},{-16,30,28},{4,-30,20}},
192 };
193
194 static int modp(long long x, int m) {
195     int r=(int)(x%m);
196     return r<0 ? r+m : r;
197 }
198
199 static long long det3(const int T[3][3]) {
200     return (long long)T[0][0]*(T[1][1]*T[2][2]-T[1][2]*T[2][1])
201         -(long long)T[0][1]*(T[1][0]*T[2][2]-T[1][2]*T[2][0])
202         +(long long)T[0][2]*(T[1][0]*T[2][1]-T[1][1]*T[2][0]);
203 }
204
205 /* Verify  $T^t A T = d^2 B$  and  $\det T = d^3$ . */
206 static int verify_family(const char *label, const int (*Ts)[3][3], int nt,
207     int d, const int A[3][3], const int B[3][3]) {
208     const long long d2=(long long)d*d;
209     int fail=0;
210     for (int k=0;k<nt;k++) {
211         long long det=det3(Ts[k]);
212         if (det<0) det=-det;
213         if (det!=(long long)d*d*d) {
214             printf("bad determinant %s[%d]\n",label,k);
215             fail++;
216         }
217         for (int i=0;i<3;i++) for (int j=0;j<3;j++) {
218             long long s=0;
219             for (int a=0;a<3;a++) for (int b=0;b<3;b++)
220                 s+=(long long)Ts[k][a][i]*A[a][b]*Ts[k][b][j];
221             if (s!=d2*B[i][j]) {
222                 printf("bad identity %s[%d] entry (%d,%d)\n",label,k,i,j);
223                 fail++;

```

```

224     }
225 }
226 }
227 printf("VERIFY %-4s matrices=%d fail=%d\n",label,nt,fail);
228 return fail;
229 }
230
231 /* Check whether one map in a family sends v to an integral vector primitive
232 at every prime dividing d. */
233 static int covered(const int (*Ts)[3][3], int nt, int d,
234                  int x, int y, int z) {
235     int primes[2], np=0;
236     if (d%2==0) primes[np++]=2;
237     if (d%3==0) primes[np++]=3;
238
239     for (int k=0;k<nt;k++) {
240         const long long v[3]={x,y,z};
241         long long num[3]={0,0,0};
242         for (int i=0;i<3;i++) for (int j=0;j<3;j++)
243             num[i]+=(long long)Ts[k][i][j]*v[j];
244
245         if (modp(num[0],d) || modp(num[1],d) || modp(num[2],d))
246             continue;
247
248         int good=1;
249         for (int ip=0;ip<np;ip++) {
250             const int p=primes[ip];
251             if (modp(num[0]/d,p)==0 && modp(num[1]/d,p)==0
252                 && modp(num[2]/d,p)==0) {
253                 good=0;
254                 break;
255             }
256         }
257         if (good) return 1;
258     }
259     return 0;
260 }
261
262 static int covered_union(int direction, int x, int y, int z) {
263     if (direction==0) {
264         if (covered(GF4, (int)(sizeof GF4/sizeof GF4[0]), 4,x,y,z)) return 1;
265         if (covered(GF12, (int)(sizeof GF12/sizeof GF12[0]),12,x,y,z)) return 1;
266         if (covered(GF36, (int)(sizeof GF36/sizeof GF36[0]),36,x,y,z)) return 1;
267     } else {
268         if (covered(FG4, (int)(sizeof FG4/sizeof FG4[0]), 4,x,y,z)) return 1;
269         if (covered(FG12, (int)(sizeof FG12/sizeof FG12[0]),12,x,y,z)) return 1;
270         if (covered(FG36, (int)(sizeof FG36/sizeof FG36[0]),36,x,y,z)) return 1;
271     }
272     return 0;
273 }
274
275 static long long test_union(int direction) {
276     const int L=216;
277     long long tested=0, fail=0;
278     for (int x=0;x<L;x++) for (int y=0;y<L;y++) for (int z=0;z<L;z++) {
279         /* A globally primitive vector is primitive at 2 and at 3. */
280         if (x%2==0 && y%2==0 && z%2==0) continue;
281         if (x%3==0 && y%3==0 && z%3==0) continue;
282         tested++;
283         if (!covered_union(direction,x,y,z)) {
284             if (fail<20)
285                 printf("UNCOVERED direction=%d residue=(%d,%d,%d)\n",
286                     direction,x,y,z);
287             fail++;
288         }
289     }
290     printf("COVER direction=%s residues=%lld fail=%lld\n",

```

```

291         direction==0 ? "G_to_F" : "F_to_G",tested,fail);
292     return fail;
293 }
294
295 int main(void) {
296     int fail=0;
297
298     /* Local equivalence certificate for the genus assertion. */
299     fail+=verify_family("T3",&T3,1,3,F,G);
300     fail+=verify_family("T4",&T4,1,4,F,G);
301
302     fail+=verify_family("GF4", GF4, (int)(sizeof GF4 /sizeof GF4[0]), 4,F,G);
303     fail+=verify_family("GF12",GF12,(int)(sizeof GF12/sizeof GF12[0]),12,F,G);
304     fail+=verify_family("GF36",GF36,(int)(sizeof GF36/sizeof GF36[0]),36,F,G);
305     fail+=verify_family("FG4", FG4, (int)(sizeof FG4 /sizeof FG4[0]), 4,G,F);
306     fail+=verify_family("FG12",FG12,(int)(sizeof FG12/sizeof FG12[0]),12,G,F);
307     fail+=verify_family("FG36",FG36,(int)(sizeof FG36/sizeof FG36[0]),36,G,F);
308
309     fail+=(int)test_union(0);
310     fail+=(int)test_union(1);
311
312     printf("RESULT %s\n",fail ? "FAIL" : "PASS");
313     return fail ? 1 : 0;
314 }

```

References

- [1] J. S. Hsia, *Regular positive ternary quadratic forms*, Mathematika **28** (1981), no. 2, 231–238. doi:10.1112/S0025579300010287.
- [2] J. Ju, *Ternary quadratic forms representing the same integers*, International Journal of Number Theory **18** (2022), no. 9, 1921–1928. doi:10.1142/S1793042122500981.
- [3] R. Oishi-Tomiyasu, *Positive-definite ternary quadratic forms with the same representations over $\mathbf{Z}$* , International Journal of Number Theory **16** (2020), no. 7, 1493–1534. doi:10.1142/S1793042120500785.
- [4] Y. Kitaoka, *Arithmetic of Quadratic Forms*, Cambridge Tracts in Mathematics, vol. 106, Cambridge University Press, Cambridge, 1993.